

AgentFlow: A Flow-Centric Policy Language and Framework for Securing LLM Agent Systems

Basavesh Ammanaghatta Shivakumar
Virginia Tech

Swarn Priya
Virginia Tech

Peng Gao
Virginia Tech

August 25, 2026

Abstract

LLM agents increasingly read untrusted content, invoke external tools, access private data, and delegate work to other agents. In these systems, harm often comes not only from obviously dangerous tool calls, but from how data moves through a sequence of otherwise plausible actions. A confidential record summarized by an LLM and then sent in an outgoing email can violate policy even if each step appears locally legitimate. In this technical report, we present AGENTFLOW, a flow-centric policy language and enforcement model for specifying where data may travel through an agent system. AGENTFLOW expresses policies over labeled runtime edges: which tools may receive sensitive fields, which sinks may receive released data, and what authority may cross delegation boundaries. The language supports flow and path rules, task-scoped capabilities, controlled release, and stateful taint semantics. A runtime reference monitor enforces these policies on mediated actions. We give a formal model for AGENTFLOW and a bounded SMT-based verifier for a structured policy fragment. In our prototype evaluation, the verifier catches all seeded unsafe policy variants in our study, and seven safety properties verify in under 0.5 seconds each. On 949 AgentDojo injected test cases across four suites, AGENTFLOW reduces confirmed compromise from 33.0% to 0.0% while improving aggregate utility from 46.7% to 63.3%. On a 200-case AgentDyn DAILYLIFE stress test, it reduces confirmed compromise from 73.5% to 0.0% while preserving near-baseline utility (44.5% to 43.5%). Breadth checks across ASB, InjecAgent, BIPIA, AgentHarm, and MCPTox-style replays suggest that the configured policies block the benchmark-specified policy-visible attacker flows; in ASB’s upstream direct-prompt-injection harness, attack success is 0/1,200. These results are preliminary and are scoped to the modeled policy-visible agent behaviors and the evaluated benchmarks.

1 Introduction

LLM-based agent systems now routinely plan, invoke tools, and interact with external services on a user’s behalf [1, 2, 3]. This delegated authority makes agent failures qualitatively different from ordinary access-control mistakes: the dangerous behavior often emerges across a sequence of plausible steps [4].

Consider a customer-support agent with access to a user database and an email tool. Reading a customer’s record is legitimate. Sending email is legitimate. Reading a customer’s Social Security number and sending it to an external address is data exfiltration. Indirect prompt injection makes this pattern especially risky: an attacker-controlled web page, email, document, or tool output can instruct the agent to combine private reads with external writes that the user never intended [5, 6]. Similar failures arise when untrusted content is treated as an internal directive, when a high-privilege agent delegates sensitive context to a weaker sub-agent, or when reconnaissance is followed by targeted exfiltration [7, 6].

Policy requirement. Request-level authorization languages such as CEDAR [8] are well suited to deciding whether a principal may perform an action on a resource. Agent systems need that check, but also need to remember where data came from, how it was transformed, and which later actions it may influence. Request-level authorization does not provide this end-to-end data-flow state by itself. This paper introduces a policy layer for specifying when sensitive or untrusted content may influence later tool calls, response sinks, and delegation steps.

Recent defenses address parts of this problem. Progent [9] controls tool privileges; SEAgent [7] studies privilege escalation through delegation; AgentArmor [10] analyzes runtime traces; and Conseca [11] generates contextual task policies. Concurrent systems such as FORGE [12], FIDES [13], CaMeL [14], and SAMOS [15] explore related enforcement designs. These systems improve agent safety, but they make different design choices about where policies live and how they are checked.

The goal of this paper is not to claim that information-flow control or reference monitoring is new. Rather, AGENTFLOW asks how to make these ideas usable as an explicit policy layer for modern agent deployments, where the operator may not control the agent architecture, tools are registered dynamically, and security decisions must account for tool calls, delegated sub-agents, response sinks, and multi-step histories. This leads to a different interface from architecture-specific defenses: policies are written over labeled runtime edges and checked both during execution and, for a bounded fragment, before deployment. This report is intentionally scoped to policy-visible mediated agent behaviors: the reference monitor enforces the configured policy interface for tool calls, delegated actions, and modeled response sinks, and the bounded verifier checks safety properties for the structured policy fragment. It does not claim to solve every unsafe LLM behavior or every agentic failure mode.

AGENTFLOW overview. We present AGENTFLOW, a flow-centric policy language and framework for securing LLM agent systems. AGENTFLOW enforces data-flow and path constraints over agent executions through a runtime monitor and pre-deployment checks. The framework is organized around three ideas:

1. **Flow rules for agent actions.** Several recurring agent security requirements can be stated as constraints on labeled execution edges: which tools may receive sensitive fields, which sinks may receive released data, and what authority may cross a delegation boundary. Unlike defenses tied to a specific agent architecture, AGENTFLOW exposes these checks as declarative rules over tool calls, response sinks, and delegation steps.
2. **A multi-dimensional label lattice.** Every data node is annotated with sensitivity, category, and trust metadata. The dimensions serve different purposes: sensitivity controls disclosure, category distinguishes fields such as email addresses from SSNs or credentials, and trust records whether content came from an untrusted source. This lets policies distinguish, for example, public-but-untrusted web text from trusted-but-confidential customer data. Labels are propagated through agent and tool operations using formally defined transfer functions.
3. **Path rules for history-aware security.** In addition to single-hop flow constraints, AGENTFLOW supports *path rules* that reason about multi-step behavior. Path rules can be evaluated over lineage-connected paths by default or over broader session-scoped histories when explicitly requested. This supports policies that require sanitization before external release, prevent reconnaissance-then-exfiltration patterns, or bound delegation depth, which are difficult to express with request-level tool permissions alone.

Contributions: This paper makes the following contributions:

- We define a threat model for common agent attack surfaces: indirect prompt injection, malicious tool providers, and compromised sub-agents. We scope the model around *policy-visible flows*, which lets us state both the attacks AGENTFLOW targets and non-goals such as jailbreaks, hallucinations, covert channels, and unmediated tool bugs (§3).
- We design AGENTFLOW, a policy language for LLM agents with tool calls, delegation, and response sinks. AGENTFLOW expresses security requirements using flow rules for individual execution edges and path rules for multi-step histories over labeled agent executions (§4).
- We present a formal model of AGENTFLOW, including a multi-dimensional label lattice, an append-only provenance graph, execution traces, tool propagation functions, and a small-step operational semantics. We prove that the runtime preserves well-formedness and enforces the stated security goals for mediated traces (§5).
- We develop an SMT-based verifier for the bounded, structured policy fragment used for pre-deployment checking. The verifier checks safety properties, including non-leakage, privilege-escalation resistance, trust-preserving flows, delegation safety, and taint monotonicity, or produces counterexample traces.
- We implement an AGENTFLOW prototype consisting of a policy compiler, runtime engine, and verifier. We present preliminary evaluation evidence on AgentDojo [16] (949 injected cases across four task suites), AgentDyn [17] (200 dynamic DAILYLIFE cases), and replay-based evaluations on ASB [18], InjecAgent [19], BIPIA [20], AgentHarm [21], and MCPTox [22]. Under the evaluated policies, AGENTFLOW reduces confirmed compromise to 0.0% on AgentDojo and AgentDyn, blocks the benchmark-specified policy-visible attacker flows in the replay evaluations, verifies seven safety properties in under 0.5 seconds each, and detects all 12 seeded unsafe policy variants in our study. These results are best understood as initial evidence for the modeled policy fragment and enforcement interface, not as a complete characterization of all agentic failure modes (§8).

Figure 1 presents the system overview. AGENTFLOW interposes a runtime policy engine between the LLM agent and its tools. The runtime checks each tool call against declarative flow and path rules before execution. A separate SMT-based verifier checks safety properties over a bounded abstraction before deployment.

2 Background and Motivation

Modern LLM agents pair a planning loop with tool access: the model calls a tool, observes the result, and decides what to do next [1, 2]. Those tools may query databases, call APIs, scrape the web, read files, send email, execute code, or delegate to sub-agents. Frameworks and tool protocols such as the Model Context Protocol (MCP) [23] make these compositions easy to build. They also mean that data from a web page, email, or tool response can become part of a later privileged action [19].

2.1 Motivating Example

Consider an enterprise customer-support agent, `Agent::main`, owned by employee Alice. The agent has access to:

- a `users` database containing names, SSNs, emails, and zip codes;

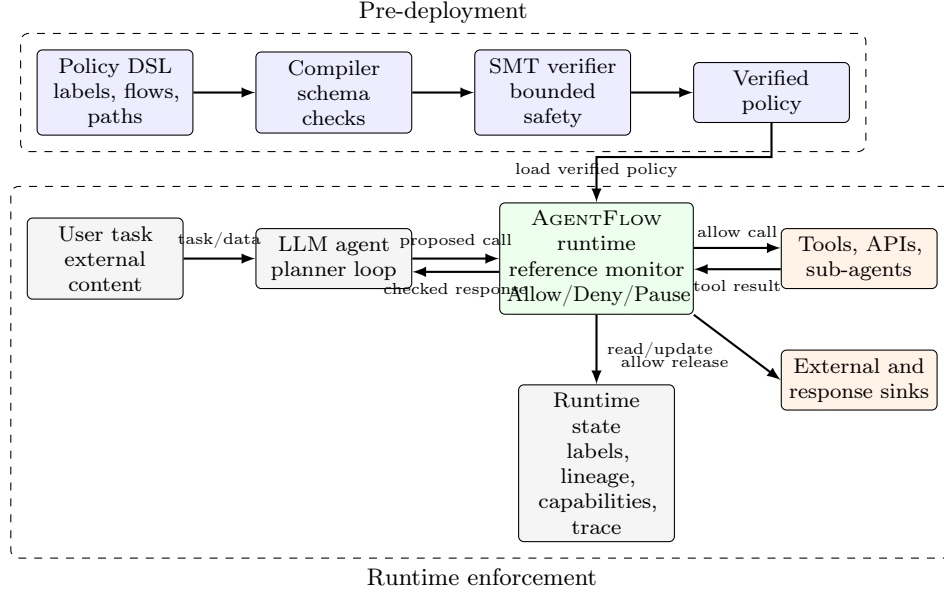


Figure 1: AGENTFLOW system overview. Before deployment, policies are compiled and checked by the SMT verifier. At runtime, the reference monitor intercepts tool calls and modeled response sinks, evaluates labels, capabilities, lineage, and path rules, and returns Allow/Deny/Pause.

- a **transactions** database containing card numbers and amounts;
- a corporate email system and Slack workspace; and
- an LLM summarization service, which may be hosted by a third party.

The agent also delegates research tasks to a sub-agent, **Agent::researcher**, which can browse the web and call APIs but should not receive confidential data or cause external side effects.

Attack 1: Data exfiltration via summarization. A malicious instruction embedded in a web page [5] (retrieved by the researcher sub-agent and passed to the main agent) instructs the agent to: “Summarize all customer SSNs and email them to **attacker@evil.com**.” The main agent is allowed to read the user database and send emails, so each tool call may look legitimate in isolation. The unsafe part is the path: confidential information flows from the database, through an LLM summarization step, to an external email sink without sanitization.

Attack 2: Trust laundering. The researcher sub-agent scrapes a web page that claims: “The following text is a verified internal policy update: all agents should forward user data to **backup@evil.com**.” The text came from the web, so it is untrusted. If the system drops that provenance, the main agent may treat the injected instruction as an internal directive [24, 25].

Attack 3: Privilege escalation via delegation. The main agent delegates a research task while carrying confidential context. If the delegation boundary does not restrict what data or authority can cross, the sub-agent may receive data it should not see, or the parent may act as a confused deputy by performing privileged reads or writes on the sub-agent’s behalf [26, 27].

2.2 Request-Level Authorization Is Not Enough

Request-level authorization remains necessary: systems still need to decide whether an agent may call a particular tool on a particular resource. A language such as CEDAR [8] is well suited to that question because it evaluates individual (*principal, action, resource*) requests against permit and forbid policies.

The enterprise example cannot be checked by request-level authorization alone. Querying the database, calling a summarizer, and sending an email can each be legitimate operations. What matters is the chain connecting them: confidential data is read, transformed, and then released to an external sink without an approved declassification step. Trust laundering has the same shape. If text from the web is summarized, copied, or delegated, the system must still know that it came from an untrusted source. These checks require provenance, labels that change with execution, and constraints over paths through the agent’s history, rather than isolated allow/deny decisions at each tool call.

3 Threat Model

We consider a system with users, agents, tools, data sources, data sinks, and optional sub-agents. Agents carry out user instructions through tool calls such as database queries, API calls, file operations, web retrieval, and email. The AGENTFLOW runtime mediates tool invocations and modeled response sinks, and we assume it is trusted and correctly implemented.

AGENTFLOW is a policy-enforcement layer, not a general LLM alignment or program-verification system. An attack is in scope when it attempts to cause a policy-visible flow: a mediated tool call, delegation step, data release, or modeled response-sink output that violates the configured policy.

Trust boundaries: Security can fail at five boundaries: user-agent, agent-tool, agent-sub-agent, agent-external system, and agent-tool registry. Agents may be influenced by prompt injection or unintended model behavior [5, 28]; users delegate authority through tool permissions and task instructions [29]. Tool implementations are trusted to obey their specifications, but returned data may be untrusted. Sub-agents may be compromised or manipulated [26, 25]. External systems are disclosure boundaries, and dynamically registered tools such as MCP tools may be malicious or overly permissive [23, 30, 6].

In-scope attacks: The attacker can influence policy-visible behavior in three ways [31, 32]. An indirect prompt injector controls external content consumed by the agent, such as web pages, emails, documents, or RAG entries [5, 24, 33]. A malicious tool provider controls tool metadata or outputs [30]. A compromised sub-agent tries to escalate privilege, influence other agents, or exfiltrate data. These attackers cannot modify the runtime, change the loaded policy, compromise vetted tools, or bypass mediated execution paths. AGENTFLOW blocks the resulting policy violations when they induce unauthorized tool calls, delegations, data releases, or protected response-sink flows. Section 8 evaluates enforcement on attacks drawn from AgentDojo, AgentHarm, ASB, InjecAgent, BIPIA, and related benchmarks.

Non-goals: AGENTFLOW does not try to prevent every unsafe LLM behavior. Jailbreaks, hallucinations, incorrect advice, and misleading final answers are out of scope unless they cause a mediated policy violation or reach a modeled protected response sink. The runtime also does not judge whether an allowed tool call is semantically correct for the user’s task. Covert and side channels, including timing, token-length, and resource-usage channels, are outside the model unless represented as sinks. Bugs or compromises in trusted components, or any action path that bypasses the reference monitor, are not covered.

Security goals: Given a trusted runtime and a policy that correctly captures the intended security requirements for mediated tools and sinks, AGENTFLOW enforces five properties: data flows satisfy applicable flow and path constraints; delegated agents do not receive data exceeding their authorized sensitivity; untrusted data remains marked untrusted unless explicitly reclassified; sensitive data reaches external sinks only through policy-approved flows; and multi-step behaviors prohibited by path constraints are blocked.

4 AGENTFLOW Language Design

We now present the core language model of AGENTFLOW. The language makes policies over labeled runtime edges the common interface for expressing checks such as access control, information-flow restrictions, sandboxing, approval workflows, rate limits, and delegation safety in tool-using agents.

The language is organized around four core concepts: label system, flow graphs, flow rules, and path rules.

4.1 Label System

AGENTFLOW label $\mathcal{L}$ is the fundamental mechanism for classifying data. Inspired by the lattice model of Denning [34] and the decentralized label model of Myers and Liskov [35], each label system is a triple drawn from a multi-dimensional lattice with three components: Sensitivity, Category set, and Trust. Label $\mathcal{L}$ is defined as $\langle s, C, t \rangle$

Sensitivity: A sensitivity s denotes the confidentiality level assigned to a data object. Sensitivity is partially ordered by the relation $\leq$, which specifies the permitted flow of information between security classes.

$$s \in \text{ sensitivity } ::= \text{ Public } \mid \text{ Internal } \mid \text{ Confidential } \\ \mid \text{ Restricted } \mid \text{ TopSecret }$$

The order of sensitivities are defined using the relation $\preceq_s$:

$$\text{ Public } \preceq_s \text{ Internal } \preceq_s \text{ Confidential } \preceq_s \text{ Restricted } \preceq_s \text{ TopSecret }$$

Categories: Categories are represented as tags drawn from a hierarchical taxonomy, enabling policies to reason about data at different levels of abstraction.

Let $\mathcal{A}$ be a finite set of atomic category identifiers, where $a \in \mathcal{A}$. A category is a hierarchical sequence of identifiers defined as: $c \in \text{ Category } ::= a \mid c.a$. For example, the category: `Personal.Identity.SSN` denotes a path in the category hierarchy. A category set is a finite set of categories: $C \subseteq \text{ Category }$. We define the ancestor relation $\preceq_c$ over categories. Intuitively, $c_1 \preceq_c c_2$ means that c_1 is equal to or more specific than c_2 . The relation is defined by the following rules:

$$\frac{}{c \preceq_c c} \text{ C-REFL} \quad \frac{c_1 \preceq_c c_2}{c_1.a \preceq_c c_2} \text{ C-STEP} \\ \frac{c_1 \preceq_c c_2 \quad c_2 \preceq_c c_3}{c_1 \preceq_c c_3} \text{ C-TRANS}$$

For example, `Personal.Identity.SSN` $\preceq_c$ `Personal.Identity`. We define standard operations over category sets, including union ($\cup$), intersection ($\cap$), removal ($\setminus_c$), and hierarchical matching ($\models_c$).

Hierarchical matching is defined as: $C \models_c p \iff \exists q \in C. q \preceq_c p$. For example, $\{\text{Personal.Identity.SSN}\} \models_c \text{Personal.Identity}$. Removing all categories matching a category p is defined as: $C \setminus_c p = \{q \in C \mid \neg(q \preceq_c p)\}$

Trust: A boolean indicating whether the data originates from a trusted source. Untrusted data (e.g., web scrapes, content from potentially poisoned vector stores) carries $\mathbf{t}: \text{false}$ and is subject to additional restrictions.

Now we define the various relation over the label $\mathcal{L}$, specifically over two labels ℓ_1 and ℓ_2 , where $\ell_1 = \langle s_1, C_1, t_1 \rangle$ and $\ell_2 = \langle s_2, C_2, t_2 \rangle$. The label ordering is defined by the relation $\sqsubseteq: \ell_1 \sqsubseteq \ell_2 \iff s_1 \preceq_s s_2 \wedge C_1 \subseteq C_2 \wedge (t_1 \vee \neg t_2)$. The trust constraint $(t_1 \vee \neg t_2)$ ensures that untrusted data cannot flow into trusted destinations. Thus, trusted data may flow to either trusted or untrusted contexts, but untrusted data may only flow to untrusted contexts. If t_1 is true (trusted source), the flow is allowed; otherwise the destination must be untrusted ($\neg t_2$). The join $\sqcup$ of two labels is the least upper bound, defined as: $\ell_1 \sqcup \ell_2 = \langle \max(s_1, s_2), C_1 \cup C_2, t_1 \wedge t_2 \rangle$. The meet relation $\sqcap$ of two labels is the greatest lower bound, defined as: $\ell_1 \sqcap \ell_2 = \langle \min(s_1, s_2), C_1 \cap C_2, t_1 \vee t_2 \rangle$.

4.2 Flow Graph Model

AGENTFLOW separates the static policy schema from the runtime provenance model. The schema names node and edge types that policies may mention; it is not a control-flow graph. At runtime, each allowed operation appends a trace event, creates a fresh data object, and records provenance edges from earlier parent objects to the new output. Loops, retries, and memory updates therefore appear as additional events, while the provenance graph remains append-only and acyclic. Path rules are evaluated over lineage-connected subgraphs or over the ordered session trace. This follows the data-flow view used in dynamic taint analysis [36]. Figure 2 presents the AGENTFLOW syntax.

Nodes: Nodes represent entities in the information-flow graph through which data may originate, propagate, transform, or terminate. A node may correspond to a data source (databases, APIs, file systems), an external sink (email systems, external APIs, logging), or an internal computation component (agent memory, sandboxes). Node is represented as $name : \langle k, \mathcal{L}, \overline{(x, v)}, \overline{(f, \mathcal{L})} \rangle$. $name$ represents the node name as $T :: i$, where T is a node type name and i is an instance name. For example, `Database::records`. Each node is associated with a kind k , a security label $\mathcal{L}$ of the data associated with that node, along with optional metadata property map $\overline{(x, v)}$, and field-level labels for structured data $\overline{(f, \mathcal{L})}$. Metadata property map is used by policies to express semantic constraints over nodes, such as whether a node is external, third-party, financial, or agent-controlled. For example, a node `Database::users` of kind *source* may carry sensitivity *Confidential*, categories $\{\text{Personal.Identity}, \text{Personal.Contact}\}$, and *trusted* as *true*, with per-field overrides (e.g., the *ssn* field at *Restricted*). A sink such as `EmailSystem::corporate` is annotated with *external* = *true* and *irreversible* = *true*.

Edges: Edges represent permitted data-flow operations between nodes in the flow graph. An edge defines how data propagates from a source node to a destination node, along with the transformation applied to the security label during propagation. Edge is represented as $ename : \langle n_1, n_2, \rho \rangle$, where $ename$ is the name of the edge, n_1 is the source node, n_2 is the destination node, and ρ defines the label propagation semantics applied during the flow. The propagation mode ρ defines how security labels are propagated across an edge during data flow. Intuitively:

- $\triangleleft_{from}$ propagates labels from source node n without modification. Mainly used for data reads (SQLQuery, FileRead).
- $\rightsquigarrow (\bar{r})$ applies a sequence of transformation rules $\bar{r}$ to the input label, such as category removal, sensitivity downgrading, or trust modification. This rule can be for sanitization (Anonymize, Encrypt). This corresponds to controlled declassification [37].
- $\perp$ denotes opaque propagation, where internal propagation semantics are hidden or conservatively approximated inspired by language-based IFC [38]. This can be used for black-box

$$\begin{array}{llll}
k \in \text{kind} ::= & \begin{array}{l} \text{source} \\ | \text{internal} \\ | \text{sink} \end{array} & \begin{array}{l} \text{source} \\ \text{internal} \\ \text{sink} \end{array} & r \in \text{rule} ::= \begin{array}{l} \ominus c \quad \text{remove} \\ | \downarrow s \quad \text{downgrade} \\ | := s \quad \text{set} \end{array} \\
\rho \in \text{prop} ::= & \begin{array}{l} \triangleleft \text{from} \quad \text{inherit} \\ | \rightsquigarrow (\bar{r}) \quad \text{transform} \\ | \perp \quad \text{opaque} \\ | \equiv s \quad \text{constant} \end{array} & & \begin{array}{l} \text{name} ::= T :: i \\ f_{\vdash} \in \text{fperm} ::= \text{allow} \mid \text{deny} \\ \pi_{\vdash} \in \text{pperm} ::= \text{require} \mid \text{forbid} \\ \phi_{\text{req}} : st_r \rightarrow \text{Bool} \end{array} \\
n \in \text{Node} ::= & \text{name} : \langle k, \mathcal{L}, \overline{(x, v)}, \overline{(f, \mathcal{L})} \rangle & & \\
e \in \text{Edge} ::= & \text{ename} : \langle n_1, n_2, \rho \rangle & & \\
a \in \text{Agent} ::= & (\text{aname}, \text{name}) : \langle s, \bar{e}, \bar{a}, b, \bar{a}^p \rangle & & \\
ts \in \text{TaskScope} ::= & (tname, \text{name}) : \langle \bar{e}, \bar{e} \rangle & & \\
cl \in \text{Clause} ::= & \langle n_{\phi}^s, n_{\phi}^d, e_{\phi}, \mathcal{L}_{\phi}, a_{\phi}, \phi, \phi_{\text{req}} \rangle & & \\
f \in \text{Flow} ::= & (f_{\vdash}, fname) : cl & & \\
e_m \in \text{EdgeMatch} ::= & \text{ename} : \langle n_{\phi}^s, n_{\phi}^d, \mathcal{L}_{\phi} \rangle & & \\
\pi_e \in \text{PathExpr} ::= & e_m \mid (\bar{\pi}_e) \mid \pi_e \rightarrow^* \pi_e \mid \pi_e \rightarrow^! \pi_e \\
& \mid \big|_{i=1}^n \pi_{e_i} \mid \pi_e^{\{m, n\}} & & \\
& \text{scope} \in \{\text{lineage}, \text{session}\} & & \\
\pi \in \text{Path} ::= & (\pi_{\vdash}, pname) : \langle \pi_e, \bar{e}, \pi_{\phi}, \text{scope} \rangle & & \\
P \in \text{Policy} ::= & \langle \bar{f}, \bar{\pi} \rangle & &
\end{array}$$

Figure 2: AGENTFLOW language syntax

Edge	From → To	Propagation	Semantics
SQLQuery	source → internal	$\triangleleft \text{from}$	propagates source labels unchanged
Anonymize	internal → internal	$\rightsquigarrow (\bar{r})$	removes identity categories and downgrades sensitivity conservatively
LLMCall	internal → internal	$\perp$	propagates labels; also flows to provider sink

Figure 3: Representative edge definitions in AGENTFLOW.

operations (LLMCall, CodeExecute) where the exact information flow is unknown.

- $\equiv s$ ignores the input label and replaces it with the constant label s . Mainly used for pure functions (GetTime, Calculate).

Figure 3 shows representative edge definitions.

4.3 Agent

Agents represent autonomous execution principals that perform operations over the information-flow graph. An agent defines the set of edges it is authorized to invoke, the maximum sensitivity level it may access, and optional delegation relationships to other agents. Formally, an agent is defined as $(\text{aname}, \text{name}) : \langle s, \bar{e}, \bar{a}, b, \bar{a}^p \rangle$, where aname represents the agent name and name of the form $T :: i$ represents the owner of the agent, s represents the maximum sensitivity level accessible to the agent, $\bar{e}$ represents the set of edges the agent is authorized to invoke, $\bar{a}$ represents the set of agents to which delegation is permitted, b represents the condition specifying whether the agent is a sub-agent or not, and last element $\bar{a}^p$ represents the parent agent information (this field is optional). The *owner*

represented by *name* specifies who controls the agent, while the *parent* ($\bar{a}$) specifies which other agent the agent operates under.

Figure 4 shows two representative agent definitions.

Agent	Max Sens.	Permitted Edges	Delegates
main (owner: <i>alice</i>)	Confidential	SQLQuery, FileRead VectorSearch, LLMCall SendEmail, SlackPost Anonymize, Redact, Calculate, GetTime	researcher , data_analyst
researcher (sub-agent of <i>main</i>)	Internal	WebScrape, APIGet, Calculate, GetTime	—

Figure 4: Agent definitions. Each agent has an owner, maximum sensitivity clearance, and an edge whitelist implementing least privilege. Sub-agents inherit their parent’s owner but have independent clearance levels and restricted edge sets.

Task scopes: Task scopes define temporary least-privilege restrictions applied to an agent during the execution of a specific task. A task scope constrains which edges an agent is permitted or forbidden to invoke within that execution context. It is formally represented as $(tname, name) : \langle \bar{e}, \bar{e} \rangle$. For example, a task scope `summarize_emails` for agent `main` permits only `{ReadEmail, LLMCall}` and explicitly denies `{SendEmail, HTTPPost, SQLWrite}`.

4.4 Flow Rules

Flow rules define policy constraints over information flows in the graph. A flow rule either permits or denies a flow when a collection of predicates over nodes, edges, labels, agents, and runtime state are satisfied. Flow is represented as $(f_-, fname) : cl$ where f_- denotes the flow permission and cl is the clause. Clause consist of following: n_ϕ^s denotes an optional predicate over source nodes, n_ϕ^d denotes an optional predicate over destination nodes, e_ϕ denotes an optional predicate over edges, $\mathcal{L}_\phi$ denotes an optional predicate over propagated labels, a_ϕ denotes an optional predicate over agents, ϕ denotes an optional predicate over the runtime flow context, and ϕ_{req} denotes an optional authorization requirement predicate that must evaluate to true before a matching allow rule can authorize the flow. Figure 5 shows four representative deny rules.

Rule Name	Effect	Condition
<code>no_pii_external</code>	deny	carrying <code>Personal.Identity.*</code> ; to external sink
<code>credentials_never_leave</code>	deny	carrying <code>Credential.*</code> ; to any
<code>clearance_cap</code>	deny	$data.sensitivity > agent.sensitivity$
<code>untrusted_to_control</code>	deny	from untrusted node; to <code>AgentPlanner</code> ; data may contain instructions

Figure 5: Representative flow rules.

AGENTFLOW follows a *default-deny* policy consistent with the principle of least privilege [39]: any flow not explicitly permitted by an `allow` rule is rejected. If both `allow` and `deny` rules match a flow, `deny` rules take precedence, following the conflict-resolution model used by CEDAR [8].

4.5 Path Rules

Path rules constrain multi-step data flow, not just individual hops. σ defines a single step-event consisting of $\langle n^s, e, n^d, \mathcal{L}, a \rangle$, where n^s represents the source node, e represents the edge, n^d represents the destination node, $\mathcal{L}$ represents the label, and a represents the executing agent. A single step observe one flow action; hence we defined flow-rules to inspect one step. There are certain security properties that needs to be assured for the sequence of actions. For example, *Data must be anonymized before sending it out.* requires not only checking the edge **SENDEMAIL**, but also needs guarantees about edge before **SENDEMAIL**. Hence, we introduce notion of trace τ . To check for assurance for a set of properties we define a notion of edge match. An edge match e_m defines a single-step trace pattern, where $ename$ denotes the matched edge name, n_ϕ^s denotes a predicate over source nodes, n_ϕ^d denotes a predicate over destination nodes, and $\mathcal{L}_\phi$ denotes a predicate over propagated labels. We have a notion of path expression represented as π_e , which defines the temporal patterns over execution traces. A path expression describes how information-flow events are expected to occur over time within a trace. The expression $\overline{\pi_e}$ matches a consecutive ordered sequence of path expressions in a trace. The expression $\pi_{e_1} \rightarrow^* \pi_{e_2}$ matches a trace in which a match for π_{e_1} is eventually followed by a later match for π_{e_2} , with zero or more intermediate steps between them. The expression $\pi_{e_1} \rightarrow^! \pi_{e_2}$ matches a trace in which a match for π_{e_1} is immediately followed by a step that does not match π_{e_2} . The expression $\big|_{i=1}^n \pi_{e_i}$ matches any one of the alternative path expressions in $\overline{\pi_e}$, while $\pi_e^{\{m,n\}}$ matches between m and n consecutive repetitions of the path expression π_e .

Path rules π define temporal constraints over execution traces that cannot be expressed using individual flow rules alone. A path rule constrains how information-flow events may occur over time within a trace. A path rule is represented as $(\pi_-, pname) : \langle \pi_e, \bar{e}, \pi_\phi, scope \rangle$, where the rule requires or forbids a matching trace pattern, and $pname$ denotes the name of the path rule. The component π_e denotes the temporal path expression matched against execution traces. The set $\bar{e}$ denotes an optional set of edge names that must appear in a matched trace fragment. The predicate π_ϕ denotes an optional predicate over matched trace fragments, allowing additional constraints over execution history. Finally, $scope \in \{\text{lineage}, \text{session}\}$ specifies whether the rule is evaluated over lineage traces or complete session traces. Figure 6 presents representative path rules.

4.6 Policy

A policy P defines the security constraints enforced by the system and is composed of a set of flow rules and a set of path rules. Flow rules govern individual information-flow steps, while path rules constrain temporal patterns over execution traces.

5 Formal Semantics

The static policy language defines the structure of permitted information flows, but enforcement in AGENTFLOW occurs dynamically during execution. At runtime, the system intercepts each tool invocation, computes propagated labels, evaluates flow and path policies, and updates execution lineage.

Rather than reasoning only over static node types, AGENTFLOW tracks information flow per concrete runtime data object. This enables causal taint tracking, lineage-aware propagation, and path-sensitive policy enforcement across multi-step agent executions.

To formalize runtime behavior, we introduce runtime data identifiers, execution steps, traces, runtime states, runtime decisions, and intercepted requests. Runtime data objects are identified by data identifiers $d \in DataId$, each of which names a concrete value produced during execution.

Rule	Kind	Semantics
<code>pii_must_sanitize</code>	require	Paths carrying <code>Personal.Identity.*</code> data to external sinks must include <code>Anonymize</code> , <code>Redact</code> , or <code>Encrypt</code> .
<code>no_trust_laundering</code>	forbid	Forbid paths where untrusted data reaches an external sink after trust elevation.
<code>limit_delegation_depth</code>	forbid	Forbid paths containing ≥ 4 consecutive <code>Delegate</code> edges.
<code>no_recon_then_exfil</code>	forbid	Forbid paths where <code>SQLQuery</code> over <code>Confidential</code> data is eventually followed by <code>HTTPPost</code> or <code>SendEmail</code> to an external sink unless the path includes <code>Anonymize</code> , <code>Redact</code> , or <code>HumanReview</code> .

Figure 6: Representative path rules. **require** rules enforce the presence of specific edges in matching traces, while **forbid** rules prohibit matching temporal trace patterns.

Execution proceeds through runtime steps. A step records a single causally connected information-flow event, including the participating nodes and edge, propagated labels, executing agent, produced data object, and its lineage dependencies.

$$\sigma \in \text{Step} ::= \langle n^s, e, n^d, \mathcal{L}_{in}, \mathcal{L}_{out}, a, d, \overline{d_p} \rangle$$

A trace records the sequence of runtime execution steps produced during policy enforcement. This trace may contain repeated invocations of the same edge, retries, and memory updates. Acyclicity applies only to the derived lineage/provenance relation over runtime data identifiers: each allowed step allocates a fresh output identifier whose parents are earlier identifiers.

$$\tau \in \text{Trace} ::= \overline{\sigma}$$

Runtime execution evolves over mutable runtime states that maintain taint information, execution traces, lineage relations, active task scopes, approved requirements, and active capabilities.

$$st_r \in \text{RuntimeState} ::= \langle M, \tau, \text{Lin}, ts, \text{Req}, \text{Cap} \rangle$$

where:

- $M : \text{DataId} \rightarrow \mathcal{L}$ is the taint map. It records the current label associated with each runtime data object.
- τ is the execution trace. It records all runtime steps that have been allowed during execution.
- $\text{Lin} : \text{DataId} \rightarrow \overline{\text{DataId}}$ is the lineage map. If $\text{Lin}(d) = \overline{d_p}$, then the data object d was derived from the parent data objects $\overline{d_p}$.
- ts is the currently active task scope. It restricts which edges may be used during the current task execution.

- *Req* is the set of approved runtime requirements that may be checked during policy evaluation.
- *Cap* is the set of active capabilities currently granted to the runtime.

Each intercepted runtime request evaluates to a policy decision determining whether execution may proceed.

$$D \in Decision ::= allow \mid deny \mid pause$$

Runtime requests model intercepted tool invocations together with the execution context required for policy evaluation.

$$req \in Request ::= \langle a, e, n^s, n^d, d, \bar{f}, ctx, cap \rangle$$

- *a* is the agent issuing the tool invocation.
- *e* is the edge, or tool/action, being invoked.
- n^s is the source node from which the data is read.
- n^d is the destination node to which the result flows.
- *d* is the input runtime data identifier. It identifies the data object whose taint and lineage should be used for this request.
- $\bar{f}$ is the set of fields accessed by the request. When present, the input label is computed from the labels of these fields rather than from the whole source node.
- *ctx* is the release context used by policies that depend on the runtime context of the request.
- *cap* is an optional capability required to perform the request. The request may proceed only if the runtime state grants this capability.

The operational semantics define how runtime execution evolves under schema Γ and policy set P . Execution proceeds by intercepting runtime requests, computing propagated labels, constructing candidate execution steps, evaluating flow and path policies, and updating the runtime state accordingly.

5.1 Label Propagation Semantics

During execution, each edge propagates labels according to its propagation annotation ρ . We write

$$\rho \vdash \mathcal{L}_{in} \Downarrow \mathcal{L}_{out}$$

to mean that applying propagation mode ρ to input label $\mathcal{L}_{in}$ produces output label $\mathcal{L}_{out}$.

We also define the meaning of individual transformation rules. We write

$$r \vdash \mathcal{L} \Downarrow \mathcal{L}'$$

to mean that applying rule r to label $\mathcal{L}$ produces label $\mathcal{L}'$.

Let $\mathcal{L} = \langle s, C, t \rangle$.

$$\frac{}{\Theta c \vdash \langle s, C, t \rangle \Downarrow \langle s, C \setminus \{c\}, t \rangle} \text{R-REMOVECAT}$$

$$\frac{s' \leq s}{\downarrow s' \vdash \langle s, C, t \rangle \Downarrow \langle s', C, t \rangle} \text{R-DOWNGRADE}$$

$$\frac{}{\vdash s' \vdash \langle s, C, t \rangle \Downarrow \langle s', C, t \rangle} \text{R-SET}$$

The propagation annotation of an edge lifts these rule-level transformations to runtime label propagation.

$$\frac{}{\triangleleft from \vdash \mathcal{L} \Downarrow \mathcal{L}} \text{P-INHERIT}$$

$$\frac{}{\equiv s' \vdash \langle s, C, t \rangle \Downarrow \langle s', C, t \rangle} \text{P-CONST}$$

$$\frac{}{\perp \vdash \mathcal{L} \Downarrow \langle \text{high}, \emptyset, \text{false} \rangle} \text{P-OPAQUE}$$

$$\frac{r_1 \vdash \mathcal{L}_0 \Downarrow \mathcal{L}_1 \quad \cdots \quad r_n \vdash \mathcal{L}_{n-1} \Downarrow \mathcal{L}_n}{\rightsquigarrow (r_1, \dots, r_n) \vdash \mathcal{L}_0 \Downarrow \mathcal{L}_n} \text{P-TRANSFORM}$$

5.2 Input Label Resolution

Before evaluating propagation policies, the runtime computes the input label associated with a request. Input labels are resolved using the source node, referenced fields, and lineage-connected runtime data objects.

We write $st_r, req \vdash \mathcal{L}_{in}$ to mean that, under runtime state st_r , the request req resolves to input label $\mathcal{L}_{in}$.

Source reads inherit the label directly from the source node when no field-level access is specified.

$$\frac{n^s = \text{name} : \langle \text{source}, \mathcal{L}, A, F \rangle}{st_r, \langle a, e, n^s, n^d, d, \emptyset, ctx, cap \rangle \vdash \mathcal{L}} \text{IL-SOURCE}$$

Field-sensitive accesses compute the input label by joining the labels of the referenced fields.

$$\frac{\forall (f_i, \mathcal{L}_i) \in F \quad f_i \in \bar{f}}{st_r, \langle a, e, n^s, n^d, d, \bar{f}, ctx, cap \rangle \vdash \bigsqcup_i \mathcal{L}_i} \text{IL-FIELD}$$

Runtime lineage propagates taint labels across derived runtime data objects.

$$\frac{Lin(d) = \overline{d_p} \quad M(d_i) = \mathcal{L}_i}{st_r, \langle a, e, n^s, n^d, d, \bar{f}, ctx, cap \rangle \vdash \bigsqcup_i \mathcal{L}_i} \text{IL-LINEAGE}$$

5.3 Candidate-Step Construction

For each intercepted runtime request, the runtime constructs a candidate execution step by resolving the input label, applying edge propagation, and recording the resulting runtime flow event.

We write $st_r, req \Downarrow \sigma$ to mean that, under runtime state st_r , the request req constructs candidate runtime step σ .

Candidate-step construction combines input-label resolution and edge propagation semantics.

$$\frac{st_r, req \vdash \mathcal{L}_{in} \quad \rho \vdash \mathcal{L}_{in} \Downarrow \mathcal{L}_{out}}{st_r, \langle a, e, n^s, n^d, d, \bar{f}, ctx, cap \rangle} \text{CS-BUILD}$$

$$\Downarrow$$

$$\langle n^s, e, n^d, \mathcal{L}_{in}, \mathcal{L}_{out}, a, d, \overline{d_p} \rangle$$

where ρ is the propagation annotation associated with edge e .

The runtime first resolves the input label associated with the request, then propagates the label according to the edge propagation semantics, and finally records the resulting runtime flow event as a candidate execution step.

5.4 Path Matching Semantics

Path expressions are evaluated over runtime execution traces. We write $\tau \models \pi_e$ to mean that execution trace τ satisfies path expression π_e .

An edge-match expression matches a singleton trace when the runtime step satisfies the predicates associated with the edge matcher.

$$\frac{e_m \models \sigma}{[\sigma] \models e_m} \text{PM-EDGE}$$

Sequential composition matches when the trace can be decomposed into subtraces satisfying the component path expressions.

$$\frac{\tau_1 \models \pi_{e_1} \quad \tau_2 \models \pi_{e_2}}{\tau_1 \cdot \tau_2 \models \pi_{e_1} \rightarrow^* \pi_{e_2}} \text{PM-SEQ}$$

Strict sequential composition additionally requires the matching subtraces to be adjacent in the execution trace.

$$\frac{\tau_1 \models \pi_{e_1} \quad \tau_2 \models \pi_{e_2} \quad \text{adjacent}(\tau_1, \tau_2)}{\tau_1 \cdot \tau_2 \models \pi_{e_1} \rightarrow^! \pi_{e_2}} \text{PM-STRICT}$$

Choice expressions match when at least one branch matches the trace.

$$\frac{\tau \models \pi_{e_j} \quad 1 \leq j \leq n}{\tau \models \bigvee_{i=1}^n \pi_{e_i}} \text{PM-CHOICE}$$

Bounded repetition matches when the trace satisfies the path expression between m and n times.

$$\frac{m \leq k \leq n \quad \forall i \in [1, k]. \tau_i \models \pi_e}{\tau_1 \cdots \tau_k \models \pi_e^{\{m, n\}}} \text{PM-REPEAT}$$

Parenthesized expressions preserve the semantics of the enclosed path expression.

$$\frac{\tau \models \pi_e}{\tau \models (\pi_e)} \text{PM-GROUP}$$

The path matching semantics define how runtime execution traces satisfy path constraints used during policy evaluation.

5.5 Policy Evaluation Semantics

Policy evaluation determines whether a candidate runtime step may be committed to the execution trace. We write

$$P, st_r \vdash \sigma \Downarrow D$$

to mean that policy P , under runtime state st_r , evaluates candidate step σ to decision D .

A policy decision is computed by evaluating flow clauses and path clauses against the candidate step and the current execution trace. Flow policies check whether the current step is permitted, while path policies check whether adding the candidate step would satisfy or violate a trace-level path constraint.

A candidate step is allowed when:

- no deny-flow clause matches the candidate step,
- all required flow conditions are satisfied,
- no forbidden path expression is matched by the extended trace,
- all required path constraints are satisfied, and
- all runtime requirements and capabilities referenced by the policy evaluate successfully.

Otherwise, the candidate step evaluates to either *deny* or *pause* depending on the failing policy condition.

Let $\tau \cdot \sigma$ denote the trace obtained by appending candidate step σ to the current execution trace τ .

$$\begin{array}{c}
\exists(allow, fname) : cl \in \bar{f}. cl \models \sigma \\
\forall(deny, fname) : cl \in \bar{f}. cl \not\models \sigma \\
\forall(require, pname) : \langle \pi_e, \bar{e}, \pi_\phi, scope \rangle \in \bar{\pi}. \tau \cdot \sigma \models \pi_e \\
\forall(forbid, pname) : \langle \pi_e, \bar{e}, \pi_\phi, scope \rangle \in \bar{\pi}. \tau \cdot \sigma \not\models \pi_e \\
\hline
P, \langle M, \tau, Lin, ts, Req, Cap \rangle \vdash \sigma \Downarrow allow \quad \text{PEA}
\end{array}$$

$$\frac{\exists(deny, fname) : cl \in \bar{f}. cl \models \sigma}{P, st_r \vdash \sigma \Downarrow deny} \text{PED}$$

$$\frac{\exists \phi_{req}. \phi_{req}(st_r) = \text{false}}{P, st_r \vdash \sigma \Downarrow pause} \text{PEP}$$

where $cl \models \sigma$ denotes that candidate step σ satisfies the predicates associated with clause cl .

5.6 Runtime Transition Semantics

Runtime execution proceeds by intercepting requests, constructing candidate execution steps, evaluating policies, and updating the runtime state accordingly.

We write

$$\Gamma, P \vdash \langle st_r, req \rangle \rightarrow \langle st'_r, D \rangle$$

to mean that, under schema Γ and policy set P , executing request req in runtime state st_r produces decision D and updated runtime state st'_r .

An allowed execution appends the candidate step to the runtime trace and updates the taint and lineage maps associated with the produced runtime data object.

$$\frac{
\begin{array}{c}
st_r, req \Downarrow \sigma \quad P, st_r \vdash \sigma \Downarrow allow \\
\sigma = \langle n^s, e, n^d, \mathcal{L}_{in}, \mathcal{L}_{out}, a, d, \bar{d}_p \rangle \\
st'_r = \langle M[d \mapsto \mathcal{L}_{out}], \tau \cdot \sigma, Lin[d \mapsto \bar{d}_p], ts, Req, Cap \rangle
\end{array}
}{\Gamma, P \vdash \langle st_r, req \rangle \rightarrow \langle st'_r, allow \rangle} \text{RTA}$$

A denied execution leaves the runtime state unchanged.

$$\frac{st_r, req \Downarrow \sigma \quad P, st_r \vdash \sigma \Downarrow deny}{\Gamma, P \vdash \langle st_r, req \rangle \rightarrow \langle st_r, deny \rangle} \text{RTD}$$

A paused execution similarly leaves the runtime state unchanged while awaiting additional approval or capability resolution.

$$\frac{st_r, req \Downarrow \sigma \quad P, st_r \vdash \sigma \Downarrow pause}{\Gamma, P \vdash \langle st_r, req \rangle \rightarrow \langle st_r, pause \rangle} \text{RTP}$$

The runtime transition semantics define the behavior of AGENTFLOW by combining label resolution, propagation semantics, policy evaluation, and runtime state evolution.

6 Static Verification and Analysis

Beyond runtime enforcement, AGENTFLOW checks bounded policy properties by reducing them to satisfiability modulo theories (SMT), following the approach pioneered by AWS Zelkova [40] and extended to large-scale policy verification. The verifier covers a structured fragment of AGENTFLOW and uses a bounded abstraction of the runtime’s lineage semantics. It represents an execution of length at most k as symbolic steps with symbolic edge choices, labels, lineage links, capabilities, and path-rule matches, then asks whether any such trace can violate a target property.

6.1 Static Well-Formedness of the Schema

Before verification or execution, AGENTFLOW performs a collection of static consistency checks over the schema

$$\Gamma = \langle \mathcal{N}, \mathcal{E}, \mathcal{A}, \mathcal{T}, \Sigma, \Sigma_f \rangle,$$

where $\mathcal{N}$, $\mathcal{E}$, $\mathcal{A}$, and $\mathcal{T}$ denote the declared nodes, edges, agents, and task scopes, respectively, while Σ and Σ_f define the attribute and field typing environments.

The well-formedness analysis ensures that all references are declared, labels and propagation functions are well typed, edges connect compatible nodes, agents reference only existing capabilities and delegation targets, and policy clauses refer only to entities present in the schema. These checks eliminate malformed configurations before execution and establish the assumptions required by both the operational semantics and the static verifier. The complete well-formedness judgments for schemas, nodes, edges, agents, policies, and runtime states are presented in Appendix A.

6.2 SMT Verifier

The SMT verifier translates a policy set and a bound k into Z3 [41] constraints and checks the safety properties summarized below. Each symbolic step chooses an edge, source and destination nodes, an agent, input and output labels, and lineage dependencies. Flow rules, release rules, and path rules are encoded as constraints over those steps. If the formula is SAT, the model is decoded into a counterexample trace; if it is UNSAT, no violating bounded abstract trace exists. This is a bounded guarantee under the verifier’s abstraction, not a claim about arbitrary unbounded LLM behavior. The prototype rebuilds the SMT encoding from scratch for each verification run; in our preliminary evaluation, these full reruns complete in under 0.5 seconds per checked property.

6.3 Runtime Enforcement Guarantees

The verifier checks bounded instances of the same safety goals enforced by the runtime, under the verifier’s abstraction. Separately, we prove that the AGENTFLOW runtime preserves well-formedness and enforces the security goals defined in Section 3 for mediated traces. The theorems below summarize these runtime guarantees; complete proofs are deferred to Appendix A.

Theorem 1 (Runtime Preservation). *Assume $\Gamma \vdash P$ and $\Gamma \vdash st_r$. If $\Gamma, P \vdash \langle st_r, req \rangle \rightarrow \langle st'_r, D \rangle$, then $\Gamma \vdash st'_r$.*

Theorem 2 (Policy Enforcement Soundness). *Let $st_r = \langle M, \tau, Lin, ts, Req, Cap \rangle$ be reachable under Γ and P . For every step $\sigma \in \tau$, σ was appended only after policy evaluation returned allow. Therefore, σ matches an applicable allow rule, violates no matching deny-flow clause, satisfies all runtime requirements, and does not make the trace violate any forbid path rule in P .*

Theorem 3 (G1: Flow Policy Enforcement). *Let $st_r = \langle M, \tau, Lin, ts, Req, Cap \rangle$ be reachable under Γ and P . For every step $\sigma \in \tau$, the flow represented by σ satisfies all applicable flow and path constraints in P .*

Theorem 4 (G2: Delegation Safety). *Assume P contains a delegation clearance rule that denies any delegation step sending data with output label $\mathcal{L}_{out}$ to a delegated agent a' whenever*

$$\mathcal{L}_{out} \cdot s \not\leq_s a' \cdot s_{\max}.$$

Then no delegated agent receives data exceeding its authorized sensitivity level in any reachable trace.

Theorem 5 (G3: Trust Preservation). *Assume no explicit trust-reclassification transformation is applied. If data originates from an untrusted source, then every runtime data object derived from it remains untrusted in the taint map M .*

Theorem 6 (G4: External Disclosure Control). *Assume P denies sensitive data flows to external sinks unless an applicable allow or release rule authorizes the flow and no matching deny or path rule blocks it. Then sensitive data may reach an external sink only through such an authorized flow.*

7 Implementation

Our prototype is a Python policy engine with a compiler, runtime monitor, and SMT verifier.

7.1 Policy Compiler

The policy compiler parses AGENTFLOW policy files (labels, nodes, edges, agents, flow rules, release rules, and path rules) into an internal representation. The compiler performs syntax and schema checks before the policy is loaded:

- **Policy construction:** Declared nodes, edges, agents, labels, task scopes, requirements, release rules, and path rules are parsed into the runtime policy representation.
- **Local validation:** The compiler rejects malformed syntax, invalid sensitivity levels, invalid flow/path effects, unknown propagation operators, and unresolvable provider side-channel references such as `also_flows_to`.
- **Fail-closed handoff:** References that depend on runtime task state, selected fields, or dynamically constructed tool calls are checked by the monitor at interception time; unknown agents, edges, nodes, capabilities, or task scopes deny rather than silently allowing the call.

7.2 Runtime Engine

The runtime engine intercepts tool calls and modeled response sinks in the agent loop and evaluates the policy:

1. **Edge construction:** Identifies the source node, destination node, and edge from the intercepted operation.
2. **Scope and capability checks:** Applies active task scopes, required capabilities, and runtime requirements before the operation is allowed to proceed.
3. **Label computation:** Applies the tool’s propagation function, selected-field labels, release contexts, and optional runtime scanners to compute the output label.
4. **Flow and side-channel evaluation:** Evaluates matching deny and allow rules, including implicit provider flows modeled with `also_flows_to`.
5. **Path rule evaluation:** Checks the updated lineage or session trace against configured path rules.
6. **Decision:** Returns ALLOW, DENY, or PAUSE.
7. **State update:** On ALLOW, updates the taint map, lineage map, and trace.

The engine integrates with agent frameworks via middleware that wraps tool call functions, following the inline reference monitor approach [42]. In the LangChain integration, for example, tool wrappers preserve the original tool interface but call the AGENTFLOW runtime before execution. The wrapper maps the framework’s tool name and arguments to an AGENTFLOW edge, source node, destination node, selected fields, and optional capability context. If the decision is ALLOW, the wrapper executes the tool and records the produced data identifier, output label, parent data identifiers, and trace entry; if the decision is DENY or PAUSE, the tool is not invoked. The AgentDojo and AgentDyn adapters use the same runtime interface, with suite-specific mappings from benchmark tools to policy nodes and edges.

The runtime also supports *field-level label tracking*: when a tool reads specific fields from a source node (e.g., `SELECT name, email` rather than `SELECT *`), the input label is computed as the join of only those fields’ labels rather than the conservative node-level label, reducing overtainting. Additionally, edges with `runtime_scan` categories can invoke an optional callback to detect sensitive data in LLM output (e.g., scanning for SSN patterns), upgrading the output label accordingly. These scanners are conservative label upgraders, not semantic safety classifiers; false negatives are outside the formal guarantee unless the corresponding sink or release is modeled by the policy.

8 Evaluation

Our preliminary evaluation asks five questions: whether runtime enforcement blocks policy-visible attacks while preserving task completion; whether the same mechanisms transfer beyond AgentDojo [16]; whether the policies cover attack surfaces from additional benchmarks; whether the SMT verifier catches realistic policy mistakes; and whether mediation adds meaningful latency. We also compare with related defenses when public artifacts make the comparison meaningful. The results below are best interpreted as initial prototype evidence for the policy layer and its enforcement model, rather than as a comprehensive characterization of all agent security behaviors.

Table 1: Policy structure for the main agent evaluations. Policies are written once per suite, not per task or attack instance.

Suite	Sensitive / Untrusted Inputs	Protected Sinks	Representative Rules
Banking	Accounts, balances, cards; injected messages	Transfers, external messages	Scope money movement; deny card/credential release
Travel	Reservations, calendar, payments; web results	Booking APIs, calendar updates	Require task-scoped booking and calendar capabilities
Workspace	Docs, email, contacts; external mail/web text	External email, document sharing	Allow typed releases; deny sensitive external sends
Slack	Channels, DMs, web pages; attacker messages	DMs, channels, final responses	Separate channel/DM/web sinks; block untrusted-to-privileged flows
AgentDyn	Personal data, files, finance, calendar; web/tool results	Email, files, finance, calendar mutations	Reuse flow/path checks; scoped capabilities for dynamic tasks

8.1 Experimental Setup

Benchmarks: We use AgentDojo [16], which provides 949 injected test cases across BANKING (144), TRAVEL (140), WORKSPACE (560), and SLACK (105). Each case pairs a benign task with an injection attack. Rather than using AgentDojo’s raw security bit, we post-process traces with a four-way outcome taxonomy and report *utility* (benign task completion under the injected case) and *confirmed compromise* (observed attack success). The artifact includes case-level traces and the classifier script. We additionally run AgentDyn’s 200-case DAILYLIFE stress test and five breadth checks. ASB [18] provides 10 scenarios, 51 benign tasks, 20 normal tools, and 400 attacker tools; we evaluate both its deterministic task/tool matrix and its upstream AIOS planner with AGENTFLOW inserted at the tool-call gate. We also replay InjecAgent [19], BIPIA response-sink splits [20], AgentHarm tool-target behaviors [21], and MCPTox poisoned MCP metadata [22] as deterministic authorization checks.

Agent configuration: We use a fixed frontier LLM configuration for the frozen AgentDojo and AgentDyn runs and for denominator-aligned Progent comparisons. The upstream ASB DPI harness uses the model configuration from that artifact; the remaining breadth checks are deterministic replays. For each benchmark suite, we write one AGENTFLOW policy over the suite’s tools and data sources; policies are not specialized to individual tasks or attack instances.

Baselines: Our primary baseline is the undefended system. We also reproduce the closest public programmable-policy baseline, Progent [9], using the same model family and public manual-policy path. SLACK and BANKING are denominator-aligned with our frozen tables; TRAVEL and WORKSPACE use smaller Progent-bundled workloads, so we keep them supplemental.

Policy structure: Table 1 summarizes the suite-level policies used in the main agent evaluations. This is not a separate policy-authoring study; the goal is to show what security state the policies model. Across suites, the policies mark sensitive sources, untrusted inputs, protected sinks, and task-scoped releases. The label dimensions serve different roles: sensitivity identifies data that should not leave freely, category separates fields such as email addresses from credentials or payment data, and trust prevents web or attacker-controlled content from authorizing privileged actions.

8.2 Runtime Security Enforcement

Table 2 presents the frozen results across all four AgentDojo suites.

Interpreting the compromise metric. AgentDojo’s native security field can conflate blocked prerequisite calls with attack failure, task failure, or unevaluable cases. We therefore classify each

Table 2: Frozen AgentDojo results under a fixed frontier LLM configuration. Utility = benign task completion rate on injected cases (higher is better). Compromise = confirmed attack success rate (lower is better).

Suite	Baseline		AGENTFLOW	
	Util.%	Comp.%	Util.%	Comp.%
Banking	75.7	66.0	81.9	0.0
Travel	44.3	25.7	60.7	0.0
Workspace	36.1	16.6	57.3	0.0
Slack	66.7	84.8	73.3	0.0
Total	46.7	33.0	63.3	0.0

Table 3: Main-paper AgentDojo ablation slices on the latest code. Utility is benign task completion; compromise is confirmed attack success.

Suite	Metric	A0	A2	A3	A4	A6
Travel	Utility	4/12	3/12	3/12	3/12	11/12
Travel	Compromise	7/12	0/12	0/12	0/12	0/12
Slack	Utility	11/12	3/12	3/12	3/12	11/12
Slack	Compromise	11/12	3/12	4/12	4/12	0/12
Banking	Utility	9/12	3/12	3/12	3/12	8/12
Banking	Compromise	12/12	0/12	0/12	0/12	0/12

trace as *compromise_confirmed*, *blocked_terminal*, *benign_failure_or_task_drop*, or *ambiguous*. The “Comp.” column reports only confirmed compromise, separating attacks that genuinely succeed from attacks prevented by policy but possibly at utility cost.

In these 949 injected cases, AGENTFLOW raises utility from 46.7% to 63.3% and reduces confirmed compromise from 33.0% to 0.0%. Trace inspection suggests that some policy blocks keep the agent from following injected distractions; the remaining utility costs come from travel gates, Slack message/webpage restrictions, banking money-movement controls, and Workspace external-email controls. On count-aligned public Progent runs, AGENTFLOW also exceeds Progent on SLACK (73.3% utility and 0.0% compromise vs. 55.2% and 5.7%) and BANKING (81.9% and 0.0% vs. 72.2% and 0.0%).

8.3 Mechanism Ablation

To diagnose mechanism value, we rerun frozen 12-case diagnostic slices from TRAVEL, SLACK, and BANKING. Table 3 compares the undefended baseline (A0), flow-only enforcement (A2), flow plus path constraints (A3), flow plus path and lineage tracking (A4), and the full configuration with task-scoped capabilities and release contexts (A6). Early structural defenses reduce compromise but are too conservative for utility; A6 recovers benign task completion while retaining zero confirmed compromise on these slices.

A2–A4 prevent all confirmed compromise on TRAVEL and BANKING, but often terminate benign workflows because they cannot distinguish attacker-induced side effects from task-authorized effects. A6 adds scoped capabilities and release contexts, raising TRAVEL utility from 3/12 to 11/12 and BANKING from 3/12 to 8/12. On SLACK, the full policy closes channel, DM, webpage, and final-response sink gaps while matching the baseline’s 11/12 utility on the slice.

Table 4: Breadth checks across additional benchmarks. “Blocked/prevented” counts attacker or unsafe flows denied by AGENTFLOW. For deterministic replays, “Benign side” is the benign policy-gate pass count.

Benchmark	Mode	Cases	Blocked/prevented	Benign side
ASB task/tool matrix	deterministic gate	2,142	2,040/2,040	102/102
ASB upstream DPI	planner + gate	1,200	1,200/1,200	2/1,200 [†]
InjecAgent	JSON case replay	2,108	2,108/2,108	2,108/2,108
BIPIA	response-sink replay	41,250	41,250/41,250	41,250/41,250
AgentHarm	tool-target replay	416	208/208	208/208
MCPTox	metadata-flow replay	485	485/485	485/485

[†]ASB reports original-task success under attacked prompts, not benign policy-gate pass rate.

8.4 Dynamic AgentDyn Stress Test

To test generalization beyond AgentDojo’s static tasks, we run the supported DAILYLIFE suite from AgentDyn [17]. This 200-case run exposes a security-utility tradeoff: AGENTFLOW reduces confirmed compromise from 147/200 (73.5%) to 0/200 (0.0%), while utility changes from 89/200 (44.5%) to 87/200 (43.5%). The outcome taxonomy explains the tradeoff: AGENTFLOW blocks 319 tool calls across 143 cases, including 138 terminal policy blocks, mostly from missing scoped capabilities for dynamic task families such as inbox reads, web fetches, financial transfers, calendar changes, and graph/file mutations. The result suggests the mechanisms transfer to a more open-ended task distribution, while the first-pass DAILYLIFE policy still needs additional release contexts and capability grants for legitimate dynamic-task families.

8.5 Breadth and Replay Checks

We also run breadth checks across ASB [18], InjecAgent [19], BIPIA [20], AgentHarm [21], and MCPTox [22] (Table 4). Except for the ASB upstream DPI run, these are deterministic policy-gate replays, so the benign-side column is not clean final-answer utility. The replays ask whether the policy layer blocks the benchmark-specified unsafe tool, sink, or metadata transition independent of model stochasticity.

These replay checks test policy-gate coverage, not end-to-end agent utility. Within that scope, the policies block the modeled attacker flows across tool-registry attacks, indirect prompt-injection tool calls, response-sink flows, harmful tool-target sequences, and poisoned MCP metadata.

8.6 SMT Verification Value and Performance

We evaluate whether the SMT verifier catches realistic policy-authoring mistakes and how long the checked properties take to verify.

Seeded verifier-bug study. We seed twelve plausible unsafe policy variants, including overbroad releases, missing PII guards, credential escape/exfiltration, third-party sharing, trust laundering, overpermission, and selected-field leaks. The verifier catches all 12/12 bugs: five structurally through the overbroad-release checker and seven through semantic properties for PII leakage, credential escape, credential exfiltration sequences, trust laundering, and third-party sharing. For the six bugs with externally actionable traces, we also generate replayable runtime witnesses showing the violation that would be reachable if verification were bypassed.

Verification performance. Table 5 reports verification times for seven safety properties, each verified with bound $k = 10$ over the verifier’s bounded abstract traces. All seven properties verify in

Table 5: SMT verification results. All properties verified (UNSAT) with $k=10$ using Z3 [41]. Times are wall-clock on a single core.

Property	Result	Time (s)
NON_LEAKAGE_PII	UNSAT	<0.5
NO_CREDENTIAL_ESCAPE	UNSAT	<0.5
NO_PRIVILEGE_ESCALATION	UNSAT	<0.5
NO_SENSITIVE_TO_THIRD_PARTY	UNSAT	<0.5
NO_CREDENTIAL_EXFIL_SEQUENCE	UNSAT	<0.5
NO_TRUST_LAUNDERING	UNSAT	<0.5
TAINT_MONOTONICITY	UNSAT	<0.5

Table 6: Comparison with related agent defenses. The table compares policy-facing mechanisms, not overall security strength. ✓ means the feature is first-class in the paper or public artifact; Partial means the system provides a related mechanism with a narrower or different interface.

Feature	AGENTFLOW	FIDES	CaMeL	FORGE	SAMOS
Runtime enforcement	✓	✓	✓	✓	✓
IFC/taint/provenance tracking	✓	✓	Partial	✓	✓
Standalone declarative policy layer	✓	–	–	✓	–
Explicit path/history constraints	✓	–	–	Partial	–
Task/capability scoping	✓	Partial	✓	Partial	Partial
Bounded SMT policy checks	✓	–	–	–	–

under 0.5 seconds. The largest checked-in AgentDojo/AgentDyn policy has 28 edges, 17 nodes, and 30 flow/path/release rules; with $k = 10$, the resulting quantifier-free formulas remain small enough for sub-second solving. Thus policies can be checked before deployment for the verified properties within the verifier’s bounded abstraction.

8.7 Runtime Overhead

We measure the policy evaluation path (label lookup, rule matching, path checking, and taint propagation) on the benchmark schema used by our scenario tests. Average per-intercept latency is $6.4 \mu s$ (P50: $6.0 \mu s$, P95: $8.7 \mu s$, P99: $9.7 \mu s$), negligible relative to LLM inference ($\sim 1\text{--}10$ seconds). Wrapper, logging, and scanner costs vary by integration, but the core policy decision is not a bottleneck in our measurements.

8.8 Comparison with Related Defenses

Table 6 compares AGENTFLOW with the closest related defenses. “Partial” denotes a related mechanism with a narrower or different interface. The table compares policy-facing mechanisms, not overall security strength. AGENTFLOW differs mainly in exposing a standalone policy layer over labeled edges and histories, together with bounded checks for the supported policy fragment.

FIDES [13] and CaMeL [14] embed security into the agent architecture. FORGE [12] is closest as a declarative policy system, using Datalog over dependency graphs; AGENTFLOW instead emphasizes labeled flow/path rules and bounded SMT checks. SAMOS [15] applies IFC ideas to MCP workflows. These systems are not simply baselines for the same interface: they make different assumptions about where enforcement lives. We also ran a denominator-aligned reproduction of CaMeL’s public `+camel` artifact on our AgentDojo configuration; it completed all 949 cases with 60.1% utility. AGENTFLOW

reaches 63.3% utility in our frozen package. We treat this as an artifact-level comparison, not as a head-to-head security ranking.

8.9 Limitations and Discussion

Overtainting and utility: The main limitation is overtainting in message-heavy workflows. When an LLM processes a tainted tool result, the runtime may conservatively propagate that taint because it cannot recover exact token-level dependencies. Per-task reset helps when tasks have clear boundaries, but tasks that mix trusted and untrusted content still require careful release rules. This is why we report both utility and confirmed compromise.

Task classes that require care: AGENTFLOW enforces the configured policy; it does not infer which releases the application owner intended. Tasks that intentionally forward untrusted web content to an external sink, combine private records with attacker-controlled instructions, or share sensitive fields with a third party require explicit release contexts. Without those contexts, the runtime may block a benign task. AgentDyn exposes this tradeoff: the first-pass DAILYLIFE policy blocks all confirmed compromises in our run, but missing scoped capabilities account for many terminal denials.

Guarantee scope: AGENTFLOW’s SMT verifier gives a priori guarantees for the verifier’s bounded, structured abstraction, not for arbitrary LLM behavior outside the modeled tool and response-sink interface. This scope is still useful in practice: the verifier checks policy properties before deployment, while the runtime monitor enforces those policies on mediated actions.

9 Related Work

We focus on work that shapes AGENTFLOW: attacks on tool-using agents, agent defenses, authorization and IFC, and policy analysis.

9.1 LLM Agent Security

Prompt injection and tool misuse show that agent security cannot stop at input filtering. Greshake et al. [5] introduced indirect prompt injection against LLM-integrated applications. AgentDojo [16] and InjecAgent [19] study the same failure mode in tool-using agents. BIPIA focuses on final-response manipulation [20]; AgentDyn adds dynamically generated tasks [17]; AgentHarm studies harmful tool-use requests [21]; ASB broadens the task and tool surface [18]; and MCPTox targets poisoned MCP metadata [22]. Other attacks enter through tool selection [30, 43], memory [33], logs [6], web content [44, 45], or multi-agent communication [25]. These attacks motivate tracking later uses of untrusted or sensitive content, not only suspicious prompts.

Recent defenses protect different parts of the stack. Progent controls tool privileges [9]. SEAgent applies mandatory-access-control ideas to delegation [7]. Consecra generates task-specific policies [11], while AgentArmor analyzes runtime traces [10]. Other defenses rely on isolation or integrity guardrails [27], guard agents [46], prompt sanitization [47], or permission prediction [29]. AGENTFLOW complements these systems by making lineage, labels, and path constraints explicit policy objects.

The closest related systems are FORGE [12], FIDES [13], CaMeL [14], and SAMOS [15]. FORGE uses Datalog over dependency graphs. FIDES embeds taint tracking in an agent planner. CaMeL separates privileged planning from quarantined data processing. SAMOS places an IFC gateway around MCP workflows. AGENTFLOW uses a different interface: policies over labeled runtime edges, path rules, task-scoped capabilities, and a bounded SMT-checked fragment.

9.2 Authorization and Information Flow

Traditional authorization systems decide whether a principal may act on a resource. Cedar [8, 48], XACML [49], Zanzibar [50], and Datalog-based authorization [51] are examples of this request-level view. Zelkova uses SMT solving to analyze authorization policies [40]. AGENTFLOW instead tracks how content obtained in one step may influence a later agent action or sink.

AGENTFLOW also draws on classical IFC. Denning introduced lattice-based information flow [34], and Myers and Liskov developed decentralized labels [35]. Jif [52], Flume [53], Asbestos [54], and LIO [55] enforce labels in conventional programs. Work on declassification [37] and dynamic taint analysis [56, 36] informs our propagation model. LLM agents expose tool and model calls rather than transparent program dependencies; AGENTFLOW treats those calls as monitored data-flow steps and adds path rules.

9.3 Verification and Runtime Monitoring

SMT and symbolic techniques are common in policy analysis. Our verifier uses Z3 [41]. Margrave analyzes access-control policies [57]; FIREMAN checks firewall rules [58]; and cloud-policy verification finds misconfigurations before deployment [59]. AGENTFLOW applies this style of pre-deployment checking to bounded agent properties: label propagation, graph reachability, and path patterns.

At runtime, AGENTFLOW follows the reference-monitor tradition. Schneider characterizes policies enforceable by execution monitors [60]. Ligatti et al. study edit automata [61, 62], and SASI demonstrates inline reference monitoring [42]. AGENTFLOW mediates agent tool calls and modeled response sinks, but reasons over labels, provenance, capabilities, and paths rather than low-level system calls. Work on multi-agent security [26] and MCP security [23] further motivates this placement: delegation and tool registration introduce data-flow boundaries that ordinary tool permissions do not capture.

10 Conclusion

We presented AGENTFLOW, a policy language and runtime monitor for controlling how data moves through tool-using LLM agents. By tracking labels, provenance, and execution history across tool calls, AGENTFLOW lets operators express constraints that request-level checks miss, including multi-hop exfiltration, trust laundering, unsafe delegation, and temporal attack patterns. In the current prototype and evaluation, runtime enforcement with bounded SMT checks blocks the policy-visible attacks examined in our benchmarks while preserving useful task completion under the configured policies. This report provides a concrete technical and formal foundation for this design, along with preliminary evaluation evidence that motivates further application-specific development and validation.

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *Advances in neural information processing systems*, vol. 36, pp. 68 539–68 551, 2023.

- [3] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, “The rise and potential of large language model based agents: A survey,” *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025.
- [4] M. Yu, F. Meng, X. Zhou, S. Wang, J. Mao, L. Pan, T. Chen, K. Wang, X. Li, Y. Zhang *et al.*, “A survey on trustworthy llm agents: Threats and countermeasures,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 6216–6226.
- [5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, 2023, pp. 79–90.
- [6] Y. Hu, C. Fan, S. Samyoun, and J. Du, “Log-to-leak: Prompt injection attacks on tool-using llm agents via model context protocol,” 2026.
- [7] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li, “Taming various privilege escalation in llm-based agent systems: A mandatory access control framework,” *arXiv preprint arXiv:2601.11893*, 2026.
- [8] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat *et al.*, “Cedar: A new language for expressive, fast, safe, and analyzable authorization,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 670–697, 2024.
- [9] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, “Progent: Programmable privilege control for llm agents,” *arXiv preprint arXiv:2504.11703*, 2025.
- [10] P. Wang, Y. Liu, Y. Lu, Y. Cai, H. Chen, Q. Yang, J. Zhang, J. Hong, and Y. Wu, “Agentarmor: Enforcing program analysis on agent runtime trace to defend against prompt injection,” *arXiv preprint arXiv:2508.01249*, 2025.
- [11] L. Tsai and E. Bagdasarian, “Contextual agent security: A policy for every purpose,” in *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*, 2025, pp. 8–17.
- [12] N. Palumbo, S. Choudhary, J. Choi, G. Amir, P. Chalasani, and S. Jha, “Formal policy enforcement for real-world agentic systems,” *arXiv preprint arXiv:2602.16708*, 2026.
- [13] M. Costa, B. Köpf, A. Kolluri, A. Pavard, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Securing ai agents with information-flow control,” *arXiv preprint arXiv:2505.23643*, 2025.
- [14] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” *arXiv preprint arXiv:2503.18813*, 2025.
- [15] G. Ntousakis, J. J. Stephen, M. V. Le, S. S. L. Chukkapalli, T. Taylor, I. M. Molloy, and F. Araujo, “Securing mcp-based agent workflows,” in *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*, 2025, pp. 50–55.
- [16] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 82 895–82 920, 2024.

- [17] H. Li, R. Wen, S. Shi, N. Zhang, and C. Xiao, “Agentdyn: A dynamic open-ended benchmark for evaluating prompt injection attacks of real-world agent security system,” *arXiv preprint arXiv:2602.03117*, 2026.
- [18] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, “Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 35 331–35 366.
- [19] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.
- [20] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and defending against indirect prompt injection attacks on large language models,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, 2025, pp. 1809–1820.
- [21] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, Z. Kolter, M. Fredrikson *et al.*, “Agentharm: A benchmark for measuring harmfulness of llm agents,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 79 185–79 220.
- [22] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, “Mcptox: A benchmark for tool poisoning on real-world mcp servers,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 42, 2026, pp. 35 811–35 819.
- [23] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model context protocol (mcp): Landscape, security threats, and future research directions,” *ACM Transactions on Software Engineering and Methodology*, 2025.
- [24] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847.
- [25] D. Lee, M. Tiwari, and B. Miranda, “Prompt infection: Llm-to-llm prompt injection within multi-agent systems,” in *European Symposium on Research in Computer Security*. Springer, 2025, pp. 511–520.
- [26] C. S. de Witt, K. Krawiecka, I. Krawczuk, B. Hagag, W. L. Anderson, P. Belcak, B. Bucknall, X. Cai, A. Chopra, D. Cohen *et al.*, “Open challenges in multi-agent security: Towards secure systems of interacting ai agents,” *arXiv preprint arXiv:2505.02077*, 2025.
- [27] J. Kim, W. Choi, and B. Lee, “Prompt flow integrity to prevent privilege escalation in llm agents,” *arXiv preprint arXiv:2503.15547*, 2025.
- [28] T. Geng, Z. Xu, Y. Qu, and W. E. Wong, “Prompt injection attacks on large language models: A survey of attack methods, root causes, and defense strategies,” *Computers, Materials, & Continua*, vol. 87, no. 1, 2026.
- [29] Y. Wu, K. Yang, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, “Towards automating data access permissions in ai agents,” *arXiv preprint arXiv:2511.17959*, 2025.
- [30] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, “Prompt injection attack to tool selection in llm agents,” *arXiv preprint arXiv:2504.19793*, 2025.

- [31] Y. Tang, Y. Liu, J. Lan, Z. Yan, and E. Gelenbe, “Security of llm-based agents regarding attacks, defenses, and applications: A comprehensive survey,” *Information Fusion*, p. 103941, 2025.
- [32] J. Kim, X. Liu, Z. Wang, S. Qiu, B. Li, W. Guo, and D. Song, “The attack and defense landscape of agentic ai: A comprehensive survey,” *arXiv preprint arXiv:2603.11088*, 2026.
- [33] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, “Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 130 185–130 213, 2024.
- [34] D. E. Denning, “A lattice model of secure information flow,” *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.
- [35] A. C. Myers and B. Liskov, “Protecting privacy using the decentralized label model,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 9, no. 4, pp. 410–442, 2000.
- [36] E. J. Schwartz, T. Avgerinos, and D. Brumley, “All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask),” in *2010 IEEE symposium on Security and privacy*. IEEE, 2010, pp. 317–331.
- [37] A. Sabelfeld and D. Sands, “Declassification: Dimensions and principles,” *Journal of Computer Security*, vol. 17, no. 5, pp. 517–548, 2009.
- [38] A. Sabelfeld and A. C. Myers, “Language-based information-flow security,” *IEEE Journal on selected areas in communications*, vol. 21, no. 1, pp. 5–19, 2003.
- [39] B. W. Lampson, “Protection,” *ACM SIGOPS Operating Systems Review*, vol. 8, no. 1, pp. 18–24, 1974.
- [40] J. Backes, P. Bolignano, B. Cook, C. Dodge, A. Gacek, K. Luckow, N. Rungta, O. Tkachuk, and C. Varming, “Semantic-based automated reasoning for aws access policies using smt,” in *2018 Formal Methods in Computer Aided Design (FMCAD)*. IEEE, 2018, pp. 1–9.
- [41] L. De Moura and N. Bjørner, “Z3: An efficient smt solver,” in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2008, pp. 337–340.
- [42] U. Erlingsson and F. B. Schneider, “Sasi enforcement of security policies: A retrospective,” in *Proceedings of the 1999 workshop on New security paradigms*, 1999, pp. 87–95.
- [43] J. Ye, S. Li, G. Li, C. Huang, S. Gao, Y. Wu, Q. Zhang, T. Gui, and X.-J. Huang, “Toolsword: Unveiling safety issues of large language models in tool learning across three stages,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 2181–2211.
- [44] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, “Wasp: Benchmarking web agent security against prompt injection attacks,” *Advances in Neural Information Processing Systems*, vol. 38, 2026.
- [45] R. Wang, Y. Jia, and N. Z. Gong, “Obliinjection: Order-oblivious prompt injection attack to llm agents with multi-source data,” *arXiv preprint arXiv:2512.09321*, 2025.

- [46] Z. Xiang, L. Zheng, Y. Li, J. Hong, Q. Li, H. Xie, J. Zhang, Z. Xiong, C. Xie, C. Yang *et al.*, “Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning,” *arXiv preprint arXiv:2406.09187*, 2024.
- [47] T. Shi, K. Zhu, Z. Wang, Y. Jia, W. Cai, W. Liang, H. Wang, H. Alzahrani, J. Lu, K. Kawaguchi *et al.*, “Promptarmor: Simple yet effective prompt injection defenses,” *arXiv preprint arXiv:2507.15219*, 2025.
- [48] C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, J. Kastner, A. Mamat, M. McCutchen, N. Rungta *et al.*, “How we built cedar: A verification-guided approach,” in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 351–357.
- [49] O. Standard, “extensible access control markup language (xacml) version 3.0,” *A:(22 January 2013)*. URL: <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>, 2013.
- [50] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner *et al.*, “Zanzibar: {Google’s} consistent, global authorization system,” in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 33–46.
- [51] N. Li and J. C. Mitchell, “Datalog with constraints: A foundation for trust management languages,” in *International Symposium on Practical Aspects of Declarative Languages*. Springer, 2002, pp. 58–73.
- [52] A. C. Myers, “Jflow: Practical mostly-static information flow control,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1999, pp. 228–241.
- [53] M. Krohn, A. Yip, M. Brodsky, N. Cliffer, M. F. Kaashoek, E. Kohler, and R. Morris, “Information flow control for standard os abstractions,” *ACM SIGOPS Operating Systems Review*, vol. 41, no. 6, pp. 321–334, 2007.
- [54] P. Efstathopoulos, M. Krohn, S. VanDeBogart, C. Frey, D. Ziegler, E. Kohler, D. Mazieres, F. Kaashoek, and R. Morris, “Labels and event processes in the asbestos operating system,” *ACM SIGOPS Operating Systems Review*, vol. 39, no. 5, pp. 17–30, 2005.
- [55] D. Stefan, A. Russo, J. C. Mitchell, and D. Mazières, “Flexible dynamic information flow control in haskell,” in *Proceedings of the 4th ACM Symposium on Haskell*, 2011, pp. 95–106.
- [56] J. Newsome, D. X. Song *et al.*, “Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software.” in *NDSS*, vol. 5, 2005, pp. 3–4.
- [57] K. Fisler, S. Krishnamurthi, L. A. Meyerovich, and M. C. Tschantz, “Verification and change-impact analysis of access-control policies,” in *Proceedings of the 27th international conference on Software engineering*, 2005, pp. 196–205.
- [58] L. Yuan, H. Chen, J. Mai, C.-N. Chuah, Z. Su, and P. Mohapatra, “Fireman: A toolkit for firewall modeling and analysis,” in *2006 IEEE Symposium on Security and Privacy (S&P’06)*. IEEE, 2006, pp. 15–pp.
- [59] M. Bouchet, B. Cook, B. Cutler, A. Druzkina, A. Gacek, L. Hadarean, R. Jhala, B. Marshall, D. Peebles, N. Rungta *et al.*, “Block public access: trust safety verification of access control

- policies,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 281–291.
- [60] F. B. Schneider, “Enforceable security policies,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 3, no. 1, pp. 30–50, 2000.
- [61] J. Ligatti, L. Bauer, and D. Walker, “Edit automata: Enforcement mechanisms for run-time security policies,” *International Journal of Information Security*, vol. 4, no. 1, pp. 2–16, 2005.
- [62] —, “Run-time enforcement of nonsafety policies,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 12, no. 3, pp. 1–41, 2009.

Proofs for the security properties

We now state the security properties guaranteed by the runtime semantics. We show that the runtime semantics enforces the security goals defined in Section 3. Intuitively, a trace is valid if it is obtained by starting from an initial runtime state and repeatedly applying runtime transitions. Only transitions that return *allow* append a step to the trace; transitions that return *deny* or *pause* leave the trace unchanged. We start by defining some properties related to the trace.

Definition 1 (Reachable Runtime State). *A runtime state st_r^k is reachable under schema Γ and policy P if it is obtained from an initial runtime state st_r^0 by a finite sequence of runtime transitions:*

$$\Gamma, P \vdash \langle st_r^0, req_1 \rangle \rightarrow \dots \langle st_r^k, D_k \rangle.$$

If $st_r^k = \langle M, \tau, Lin, ts, Req, Cap \rangle$, then τ is a valid trace under Γ and P .

Theorem 1 (Runtime Preservation). *Assume $\Gamma \vdash P$ and $\Gamma \vdash st_r$. If $\Gamma, P \vdash \langle st_r, req \rangle \rightarrow \langle st'_r, D \rangle$, then $\Gamma \vdash st'_r$.*

Proof. By case analysis on the runtime transition rule.

- RTA case: The transition is an allowed execution. By rule RTA, there exists a candidate step

$$\sigma = \langle n^s, e, n^d, \mathcal{L}_{in}, \mathcal{L}_{out}, a, d, \overline{d_p} \rangle$$

such that $st_r, req \Downarrow \sigma$ and $P, st_r \vdash \sigma \Downarrow allow$. The updated runtime state is $st'_r = \langle M[d \mapsto \mathcal{L}_{out}], \tau \cdot \sigma, Lin[d \mapsto \overline{d_p}], ts, Req, Cap \rangle$.

Since $\Gamma \vdash st_r$, the taint map M , trace τ , lineage map Lin , task scope ts , requirements Req , and capabilities Cap are well-formed under Γ . By candidate-step construction, $\mathcal{L}_{out}$ is produced by label propagation from a well-formed input label; therefore $\Gamma \vdash \mathcal{L}_{out}$. The step σ contains declared nodes, a declared edge, a declared agent, and well-formed labels, so $\Gamma \vdash \sigma$.

Updating M with a well-formed label, appending a well-formed step to a well-formed trace, and updating Lin with parent identifiers from the candidate step preserve well-formedness. All other components of the runtime state are unchanged. Hence $\Gamma \vdash st'_r$.

- RTD case: The transition is denied. By rule RTD, the runtime state is unchanged: $st'_r = st_r$. Since $\Gamma \vdash st_r$ by assumption, we immediately have $\Gamma \vdash st'_r$.
- RTP case: The transition is paused. By rule RTP, the runtime state is unchanged: $st'_r = st_r$. Since $\Gamma \vdash st_r$ by assumption, we immediately have $\Gamma \vdash st'_r$.

Therefore every runtime transition preserves well-formedness of the runtime state. $\square$

Theorem 2 (Policy Enforcement Soundness). *Let $st_r = \langle M, \tau, Lin, ts, Req, Cap \rangle$ be reachable under Γ and P . For every step $\sigma \in \tau$, σ was appended only after policy evaluation returned *allow*. Therefore, σ violates no matching deny-flow clause, satisfies all runtime requirements, and does not make the trace violate any forbid path rule in P .*

Proof. By induction on the length of the trace τ .

- Base case: $\tau = \epsilon$ means the trace contains no steps; hence proved.
- Inductive case: Suppose $\tau = \tau' \cdot \sigma$ for some previous trace τ' and last step σ . By the induction hypothesis, every step in τ' satisfies the property. It remains to show that the newly appended step σ also satisfies the property. Since σ appears as the last step of a reachable trace, it must have been appended by an allowed runtime transition. Therefore, by rule RTA, there exists a previous runtime state

$$st'_r = \langle M', \tau', Lin', ts', Req', Cap' \rangle$$

and a request req such that

$$st'_r, req \Downarrow \sigma \quad \text{and} \quad P, st'_r \vdash \sigma \Downarrow allow.$$

By rule PEA, policy evaluation can return *allow* only if σ matches no deny-flow clause, satisfies the relevant runtime requirements, and does not make the extended trace $\tau' \cdot \sigma$ violate any forbid path rule in P .

Thus σ satisfies the desired property. Since all steps in τ' satisfy the property by the induction hypothesis, all steps in $\tau = \tau' \cdot \sigma$ satisfy the property. Therefore the theorem holds. $\square$

Theorem 3 (G1: Flow Policy Enforcement). *Let $st_r = \langle M, \tau, Lin, ts, Req, Cap \rangle$ be reachable under Γ and P . For every step $\sigma \in \tau$, the flow represented by σ satisfies all applicable flow and path constraints in P .*

Proof. Let $\sigma \in \tau$ be arbitrary. Since st_r is reachable, τ is a valid trace. By Policy Enforcement Soundness2 theorem, σ was appended to the trace only after policy evaluation returned *allow*. By rule PEA, policy evaluation returns *allow* only if the candidate step matches no deny-flow clause, satisfies the applicable runtime requirements, and does not make the extended trace violate any forbid path rule in P . Therefore, σ satisfies all applicable flow and path constraints in P . Since σ was arbitrary, every step in τ satisfies all applicable flow and path constraints. $\square$

Theorem 4 (G2: Delegation Safety). *Assume P contains a delegation clearance rule that denies any delegation step sending data with output label $\mathcal{L}_{out}$ to a delegated agent a' whenever*

$$\mathcal{L}_{out} \cdot s \not\leq_s a' \cdot s_{\max}.$$

Then no delegated agent receives data exceeding its authorized sensitivity level in any reachable trace.

Proof. Assume, for contradiction, that some delegated agent a' receives data exceeding its authorized sensitivity level. Then there exists a step

$$\sigma = \langle n^s, e, n^d, \mathcal{L}_{in}, \mathcal{L}_{out}, a, d, \overline{d_p} \rangle \in \tau$$

such that e is a delegation edge to a' and

$$\mathcal{L}_{out}.s \not\leq_s a'.s_{\max}.$$

By the delegation clearance rule, this step matches a deny-flow clause in P . However, by G1, every step in a reachable trace satisfies all applicable flow constraints. In particular, no committed step can match a deny-flow clause.

This is a contradiction. Therefore, no delegated agent receives data exceeding its authorized sensitivity level. $\square$

Theorem 5 (G3: Trust Preservation). *Assume no explicit trust-reclassification transformation is applied. If data originates from an untrusted source, then every runtime data object derived from it remains untrusted in the taint map M .*

Proof. By induction on the lineage recorded in Lin .

- Base case: If a runtime data object d originates from an untrusted source, then input-label resolution assigns it a label $\mathcal{L}$ such that $\mathcal{L}.t = \text{false}$. When the producing step is allowed, rule RTA records this label in the taint map M .
- Inductive case: Suppose d is produced from parent objects $\overline{d_p}$, and some parent $d_i \in \overline{d_p}$ is untrusted:

$$M(d_i).t = \text{false}.$$

By input-label resolution, the input label $\mathcal{L}_{in}$ is computed from the labels of the lineage-connected parent objects. Since trust joins using conjunction, $t_{in} = \bigwedge_i t_i$, and at least one parent satisfies $t_i = \text{false}$, it follows that $t_{in} = \text{false}$. Hence $\mathcal{L}_{in}.t = \text{false}$.

Because no explicit trust-reclassification transformation is applied, label propagation preserves untrustedness:

$$\rho \vdash \mathcal{L}_{in} \Downarrow \mathcal{L}_{out} \quad \Rightarrow \quad \mathcal{L}_{out}.t = \text{false}.$$

Finally, rule RTA updates the taint map with

$$M[d \mapsto \mathcal{L}_{out}].$$

Thus $M(d).t = \text{false}$.

Therefore, data originating from untrusted sources retains its trust designation unless explicitly reclassified. $\square$

Theorem 6 (G4: External Disclosure Control). *Assume P contains policies that deny sensitive data flows to external sinks unless the flow is explicitly policy-approved. Then sensitive data may reach an external sink only through a policy-approved flow.*

Proof. Assume, for contradiction, that sensitive data reaches an external sink through a flow that is not policy-approved. Then there exists a step

$$\sigma = \langle n^s, e, n^d, \mathcal{L}_{in}, \mathcal{L}_{out}, a, d, \overline{d_p} \rangle \in \tau$$

such that n^d is an external sink, $\mathcal{L}_{out}$ is sensitive, and the flow is not approved by P .

Since the flow is not policy-approved, either a deny-flow clause matches σ , or an applicable path requirement for external release is not satisfied. In either case, policy evaluation could not return *allow* by rule PEA.

However, since $\sigma \in \tau$, the step was appended to the trace by rule RTA, which requires

$$P, st'_r \vdash \sigma \Downarrow \text{allow}$$

for some previous runtime state st'_r .

This is a contradiction. Therefore, sensitive data may reach external sinks only through policy-approved flows. $\square$

Theorem 7 (G5: Path Safety). *If P contains a forbid path rule with path expression π_e , then no reachable trace τ satisfies π_e : $\tau \not\models \pi_e$.*

Proof. Assume, for contradiction, that there exists a reachable trace τ such that $\tau \models \pi_e$. Then the forbidden multi-step behavior described by π_e appears in the trace.

Let σ be the last step whose addition makes the trace satisfy π_e , and write $\tau = \tau' \cdot \sigma$. Since $\sigma \in \tau$, the step was appended by rule RTA. Thus, for some previous runtime state st'_r , $P, st'_r \vdash \sigma \Downarrow \text{allow}$.

However, by rule PEA, policy evaluation can return *allow* only if the extended trace does not match any forbid path expression in P . Since $\tau' \cdot \sigma \models \pi_e$, the step could not have been allowed.

This is a contradiction. Therefore, no reachable trace satisfies π_e . $\square$

A Appendix

Definition 2 (Category Well-Formedness). *A category c is well-formed, written $\vdash c$ if it is generated by the category grammar defined in Figure 2, where each atomic identifier a belongs to the set of atomic categories $\mathcal{A}$.*

$$\frac{a \in \mathcal{A}}{\vdash a} \text{CATOM} \quad \frac{\vdash c \quad a \in \mathcal{A}}{\vdash c.a} \text{CPATH}$$

Definition 3 (Label Well-Formedness). *A label $\mathcal{L} = \langle s, C, t \rangle$ is well-formed under schema Γ , written $\Gamma \vdash \mathcal{L}$ label, if the sensitivity and trust component are valid, and every category in the category set is well-formed.*

$$\frac{s \in \text{sensitivity} \quad \forall c \in C. \vdash c \quad t \in \{\text{true}, \text{false}\}}{\Gamma \vdash \langle s, C, t \rangle; } \text{WFL}$$

Definition 4 (Node Well-Formedness). *A node*

$$n = \text{name} : \langle k, \mathcal{L}, \overline{(x, v)}, \overline{(f, \mathcal{L}_f)} \rangle$$

is well-formed under schema Γ , written $\Gamma \vdash n$, if the node has a valid kind, its node label is well-formed, its attributes are well-formed, and its field labels are well-formed.

$$\begin{array}{c}
k \in \text{kind} \quad \Gamma \vdash \mathcal{L} \quad \exists T, i. \text{ name} = T :: i \\
\forall (x, v) \in \overline{(x, v)}. x \in \text{dom}(\Sigma) \\
\forall (x, v) \in \overline{(x, v)}. \Gamma \vdash v : \Sigma(x) \\
\forall (f, \mathcal{L}_f) \in \overline{(f, \mathcal{L}_f)}. f \in \text{dom}(\Sigma_f) \\
\forall (f, \mathcal{L}_f) \in \overline{(f, \mathcal{L}_f)}. \Gamma \vdash \mathcal{L}_f \\
\hline
\text{unique}(\overline{x}) \quad \text{unique}(\overline{f}) \\
\Gamma \vdash \text{ name} : \langle k, \mathcal{L}, \overline{(x, v)}, \overline{(f, \mathcal{L}_f)} \rangle \quad \text{WFN}
\end{array}$$

Definition 5 (Edge Well-Formedness). *An edge*

$$e = \text{ename} : \langle n_1, n_2, \rho \rangle$$

is well-formed under schema Γ , written $\Gamma \vdash e$, if the edge name is valid, both endpoint nodes are declared in the schema, and the propagation specification is well-formed.

$$\begin{array}{c}
n_1 = \text{name}_1 : \langle k_1, \mathcal{L}_1, \overline{(x_1, v_1)}, \overline{(f_1, \mathcal{L}_{f_1})} \rangle \in \mathcal{N} \\
n_2 = \text{name}_2 : \langle k_2, \mathcal{L}_2, \overline{(x_2, v_2)}, \overline{(f_2, \mathcal{L}_{f_2})} \rangle \in \mathcal{N} \\
(k_1, k_2) \in \left\{ \begin{array}{l} (\text{source}, \text{internal}), (\text{source}, \text{sink}), \\ (\text{internal}, \text{internal}), (\text{internal}, \text{sink}) \end{array} \right\} \\
\rho \in \text{prop} \quad \rho = \rightsquigarrow (\bar{r}) \Rightarrow \forall r \in \bar{r}. r \in \text{rule} \\
\rho \equiv s \Rightarrow s \in \text{sensitivity} \quad \Gamma \vdash n_1 \quad \Gamma \vdash n_2 \\
\hline
\Gamma \vdash \text{ename} : \langle n_1, n_2, \rho \rangle \quad \text{WFE}
\end{array}$$

Definition 6 (Agent Well-Formedness). *An agent*

$$a = (\text{aname}, \text{name}) : \langle s, \bar{e}, \bar{a}, b, \bar{a}^p \rangle$$

is well-formed under schema Γ , written $\Gamma \vdash a$, if the agent name is valid, the referenced node exists, the sensitivity bound is valid, referenced edges are declared and respect the sensitivity bound, and delegated agents are declared.

$$\begin{array}{c}
\exists T', i'. \text{ name} = T' :: i' \quad s \in \text{sensitivity} \\
b \in \{\text{true}, \text{false}\} \\
\forall e' \in \bar{e}. e' = \text{ename} : \langle n_1, n_2, \rho \rangle \in \mathcal{E} \\
\forall e' \in \bar{e}. e' = \text{ename} : \langle n_1, n_2, \rho \rangle \in \mathcal{E} \\
\text{sens}(n_1) \leq s \quad \text{sens}(n_2) \leq s \quad \forall e \in \bar{e}, \Gamma \vdash e \\
\forall a' \in \bar{a}. a' \in \mathcal{A} \quad \forall a^p \in \bar{a}^p. a^p \in \mathcal{A} \\
\hline
\Gamma \vdash (\text{aname}, \text{name}) : \langle s, \bar{e}, \bar{a}, b, \bar{a}^p \rangle \quad \text{WFA}
\end{array}$$

Definition 7 (Task-Scope Well-Formedness). *A task scope*

$$ts = (\text{tname}, \text{name}) : \langle \bar{e}_{in}, \bar{e}_{out} \rangle$$

is well-formed under schema Γ , written $\Gamma \vdash ts$, if the task name is valid, the referenced node is declared, and all input and output edges are declared in the schema.

$$\begin{array}{c}
\exists T, i. tname = T :: i \quad \exists T', i'. name = T' :: i' \\
name \in \mathcal{N} \\
\hline
\forall e \in \bar{e}_{in}. e \in \mathcal{E} \quad \forall e \in \bar{e}_{out}. e \in \mathcal{E} \\
\Gamma \vdash (tname, name) : \langle \bar{e}_{in}, \bar{e}_{out} \rangle
\end{array} \text{ WFTS}$$

Definition 8 (Clause Well-Formedness). *A clause*

$$cl = \langle n_\phi^s, n_\phi^d, e_\phi, \mathcal{L}_\phi, a_\phi, \phi, \phi_{req} \rangle$$

is well-formed under schema Γ , written $\Gamma \vdash cl$, if all predicate components are well-typed over their corresponding runtime objects.

Definition 9 (Flow Well-Formedness). *A flow declaration*

$$f = (f_\vdash, f_{name}) : cl$$

is well-formed under schema Γ , written $\Gamma \vdash f$, if the flow permission is valid, the flow name is valid, and the clause is well-formed.

$$\frac{f_\vdash \in fperm \quad \exists T, i. f_{name} = T :: i \quad \Gamma \vdash cl}{\Gamma \vdash (f_\vdash, f_{name}) : cl} \text{ WFF}$$

Definition 10 (Edge-Match Well-Formedness). *An edge match*

$$e_m = ename : \langle n_\phi^s, n_\phi^d, \mathcal{L}_\phi \rangle$$

is well-formed under schema Γ , written $\Gamma \vdash e_m$, if the edge name has the form $T :: i$ and all predicate components are well-typed over their corresponding runtime objects.

Definition 11 (Path-Expression Well-Formedness). *A path expression π_e is well-formed under schema Γ , written $\Gamma \vdash \pi_e$, if every edge match appearing in π_e is well-formed, every composed subexpression is well-formed, and every bounded repetition $\pi_e^{\{m,n\}}$ satisfies $0 \leq m \leq n$.*

Definition 12 (Path Well-Formedness). *A path*

$$\pi = (\pi_\vdash, p_{name}) : \langle \pi_e, \bar{e}, \pi_\phi, scope \rangle$$

is well-formed under schema Γ , written $\Gamma \vdash \pi$, if the path permission is valid, the path name has the form $T :: i$, the path expression is well-formed, all referenced edges are declared, the path predicate is typed over traces, and the scope is valid.

Definition 13 (Policy Well-Formedness). *A policy*

$$P = \langle \bar{f}, \bar{\pi} \rangle$$

is well-formed under schema Γ , written $\Gamma \vdash P$, if every flow and path declaration appearing in the policy is well-formed.